

DSI: AI Development Program

Part 1: Introduction to AI Development

University of Chicago

Who are we?

David Jacobson

- Director of Data Science & Engineering, Data Science Institute
- 12 years leading industry data science & software teams in marketing, healthcare, and cancer therapeutics



Jim Pivarski

- Physics PhD, LHC → Data Science
- Now a member of UChicago Data Science Institute
- Builds AI workflows involving LLMs and ground-up ML models for social good



Outline

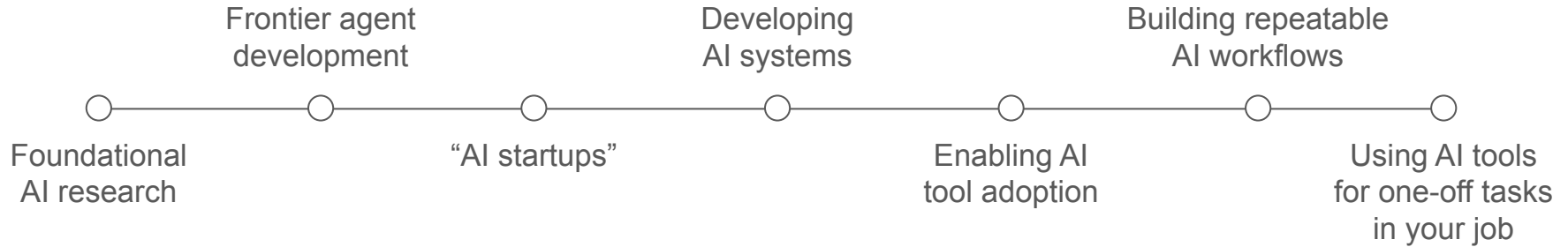
1. Motivation and Learning Objectives
2. Foundations
3. Terminology
4. Context Engineering
5. Pricing
6. Hands-on Exercise

Follow along and get exercises here:

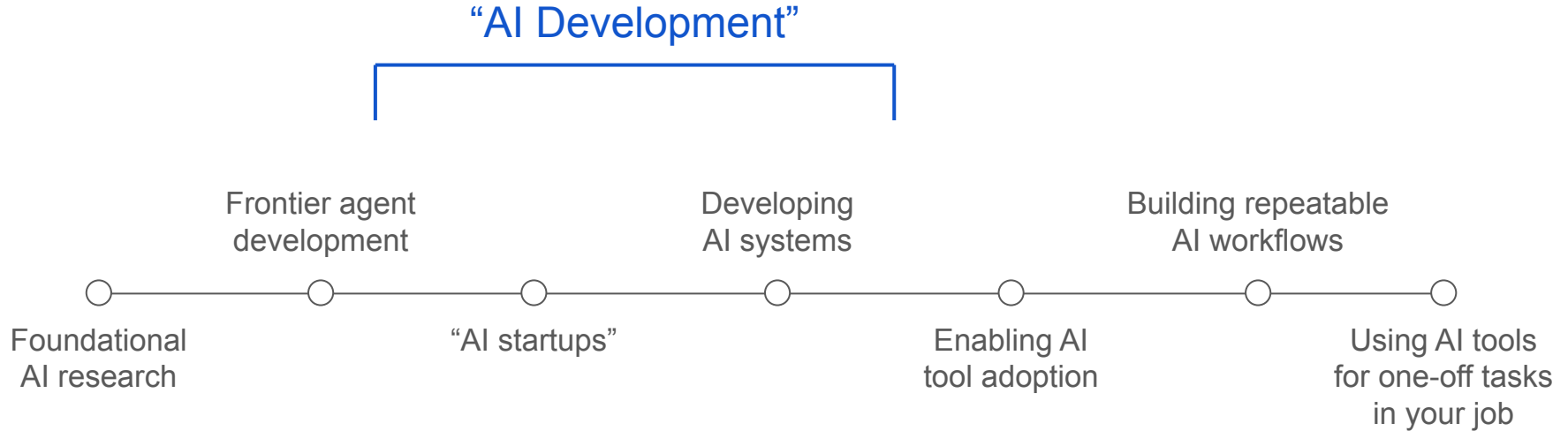
<https://github.com/dsi-rse/careers-workshop-ai-development/>

Motivation and Learning Objectives

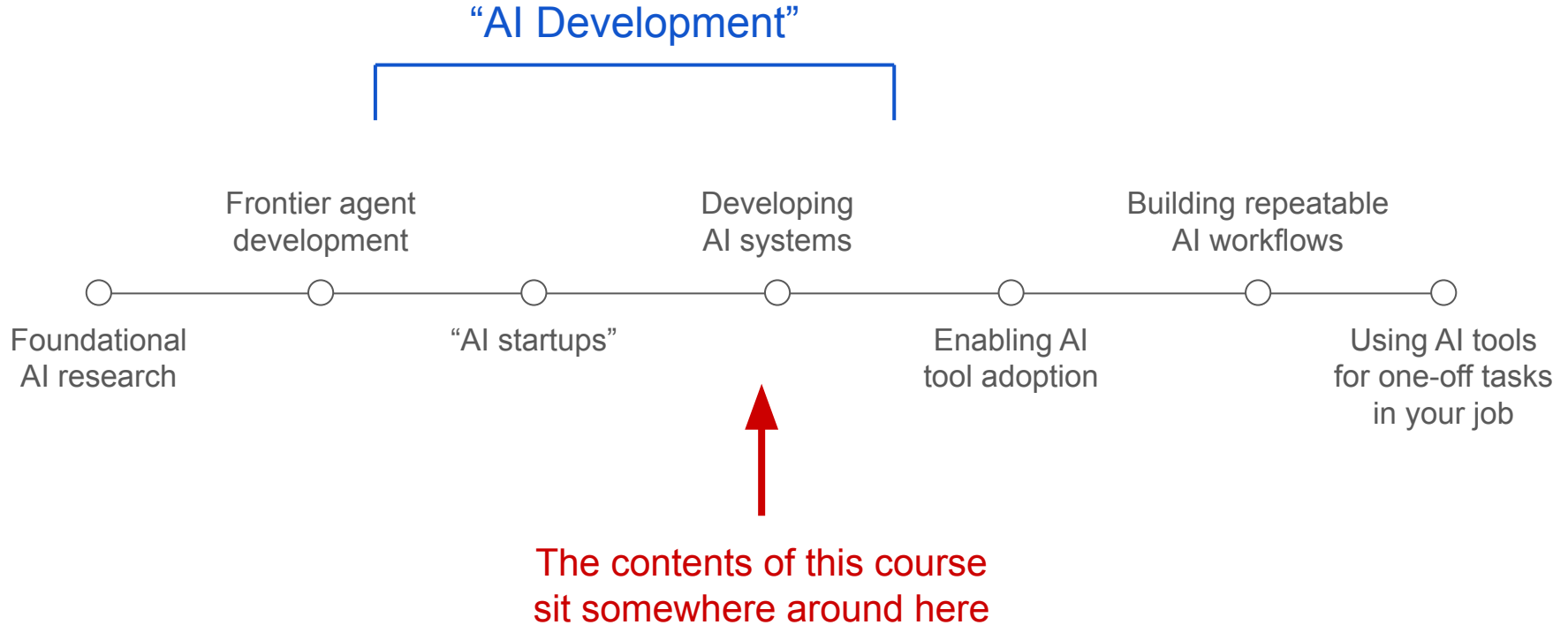
What are “careers in AI”?



Our focus in this course is on “AI Development”



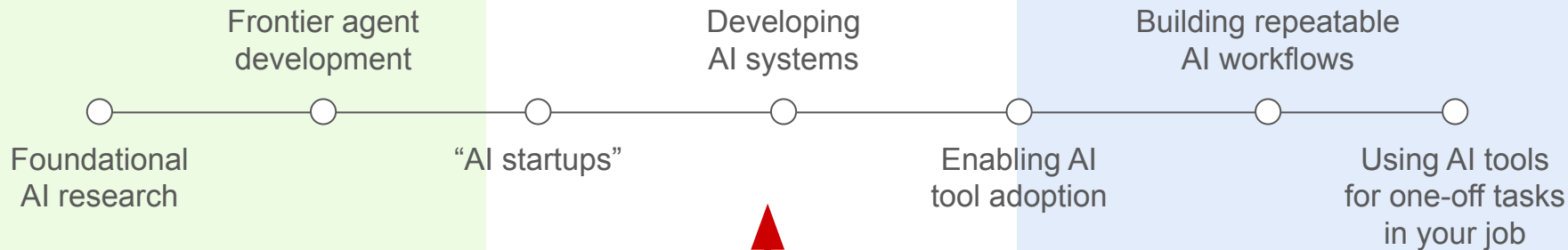
Our focus in this course is on “AI Development”



Why AI Development?

Highly dependent on the cutting edge of frontier technology and concentrated in frontier labs and academia

Highly dependent on the state of existing tools; learned by extensive practice and perpetual curiosity about practical applications

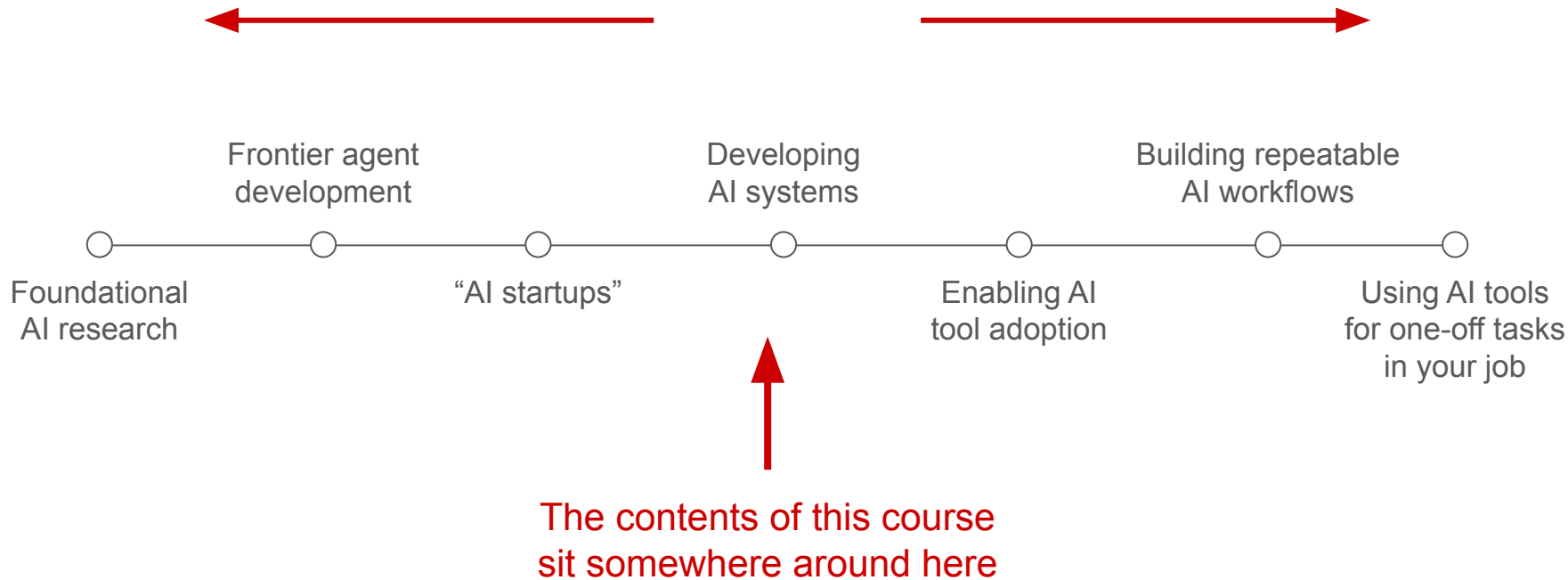


The contents of this course sit somewhere around here

Why AI Development?

Pushing the limits of AI tools is a good entryptoint into the challenges at the frontier of AI technology

Using AI systematically helps you understand it better and use it more effectively for ad hoc work.



Learning objectives

By the end of this course, you will...

1. Understand AI terminology

- Speak confidently about LLMs, tokens, context windows, agents, tools

2. Gain familiarity with trade-offs and pitfalls

- Understand what makes ROI sense and what doesn't
- Calculate and optimize token costs

3. Experience building AI systems

- Have concrete experience to talk about in job interviews
- Tune and optimize real systems
- Understand trade-offs in building such systems

4. Understand the agent loop

Today: Workshop 1

- Focus on definitions and how to work with models
- Build foundation for hands-on work
- About 45 hour of lecture, 30 minutes hands-on work, 15 min discussion at the end

Next Workshops:

- Workshops 2–4: specific implementation that highlights concrete problems
 - About 30 minutes of lecture, 1 hour hands-on
- Dates: Tuesday and Thursday this week and next

Foundations

What is AI?

Over the past 70 years, "Artificial Intelligence" has meant many different things.

What is AI?

Over the past 70 years, "Artificial Intelligence" has meant many different things.

Today, we usually mean **foundation models**:

- **Large Language Models (LLMs):** ChatGPT, Claude, Llama
- **Image/video generation:** Stable Diffusion, Sora
- **Multimodal:** Gemini, GPT-4o
- **Vision (non-generative):** CLIP, DINOv2, SAM
- **Speech:** Whisper
- **Biology and science:** ESM-2/ESM3, BioCLIP 2, AlphaEarth Foundations
- **Time series and weather:** Chronos, Aurora
- **Robotics:** RT-2, $\pi 0$

What is AI?

Over the past 70 years, "Artificial Intelligence" has meant many different things.

Today, we usually mean **foundation models**:

- **Large Language Models (LLMs):** ChatGPT, Claude, Llama
- **Image/video generation:** Stable Diffusion, Sora
- **Multimodal:** Gemini, GPT-4o
- **Vision (non-generative):** CLIP, DINOv2, SAM
- **Speech:** Whisper
- **Biology and science:** ESM-2/ESM3, BioCLIP 2, AlphaEarth Foundations
- **Time series and weather:** Chronos, Aurora
- **Robotics:** RT-2, $\pi 0$

Foundation models are large (many parameters), trained with large datasets, and capable of general tasks—not special-purpose like Machine Learning (ML).

What is AI?

Over the past 70 years, "Artificial Intelligence" has meant many different things.

Today, we usually mean **foundation models**:

- **Large Language Models (LLMs):** ChatGPT, Claude, Llama
- **Image/video generation:** Stable Diffusion, Sora
- **Multimodal:** Gemini, GPT-4o
- **Vision (non-generative):** CLIP, DINOv2, SAM
- **Speech:** Whisper
- **Biology and science:** ESM-2/ESM3, BioCLIP 2, AlphaEarth Foundations
- **Time series and weather:** Chronos, Aurora
- **Robotics:** RT-2, $\pi 0$

Foundation models are large (many parameters), trained with large datasets, and capable of general tasks—not special-purpose like Machine Learning (ML).

We'll be working with LLMs in these workshops.

An LLM-based chatbot consists of two parts:

- An **LLM**, which is a compressed representation of statistical patterns in a huge set of human-authored words
- A **transcript** of the conversation so far

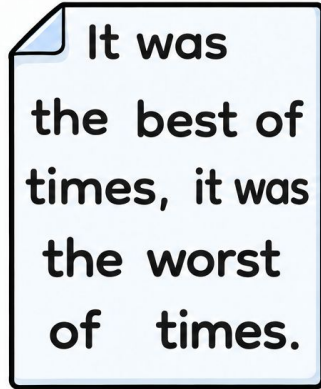
An LLM-based chatbot consists of two parts:

- An **LLM**, which is a compressed representation of statistical patterns in a huge set of human-authored words
 - "Lossy compression": the original words can't be reconstructed, only statistical patterns
- A **transcript** of the conversation so far

An LLM-based chatbot consists of two parts:

- An **LLM**, which is a compressed representation of statistical patterns in a huge set of human-authored words
 - "Lossy compression": the original words can't be reconstructed, only statistical patterns
 - The LLM is a *constant* through the conversation, unchanged by what is said
- A **transcript** of the conversation so far

First, what is "lossy compression"?



It was
the best of
times, it was
the worst
of times.

Uncompressed



It was
the best of
times, it was
the worst
of times.

**Lossless
compression**



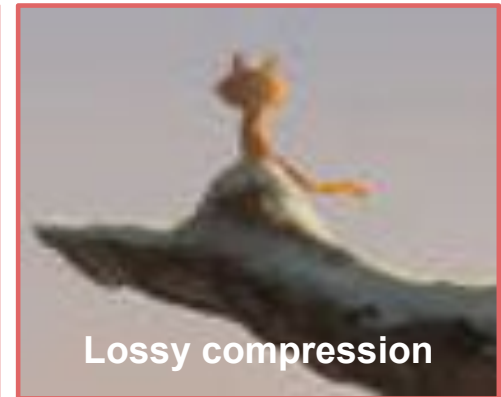
twat
he
est
oft
ime
l
it
was
wor
of
im

I s t h b e

**Lossy
compression**

Example: JPEG

JPEG files are smaller than PNG because they don't save every pixel.

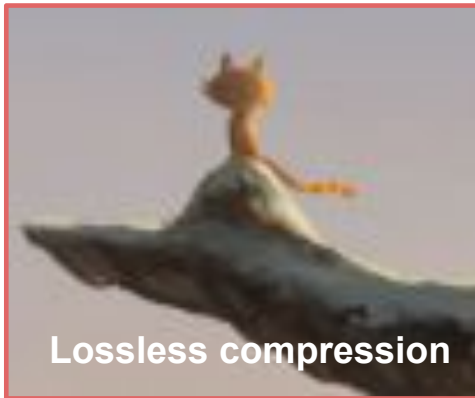


(Thanks for the analogy, Ted Chiang!)

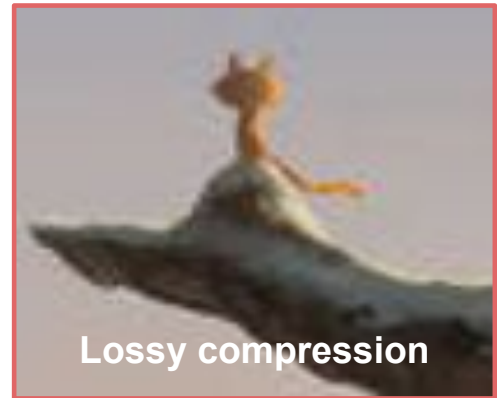
Example: JPEG

JPEG files are smaller than PNG because they don't save every pixel.

They contain statistical distributions of regions of pixels.



Lossless compression



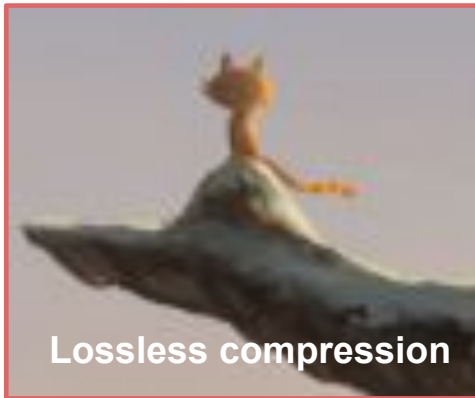
Lossy compression

Example: JPEG

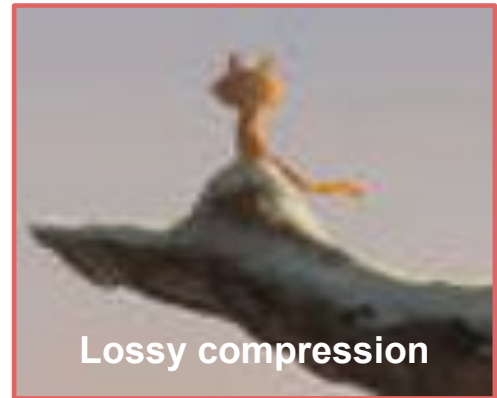
JPEG files are smaller than PNG because they don't save every pixel.

They contain statistical distributions of regions of pixels.

When reconstructed, the large-scale image features look right, but fine details have wave-like artifacts, especially around edges.

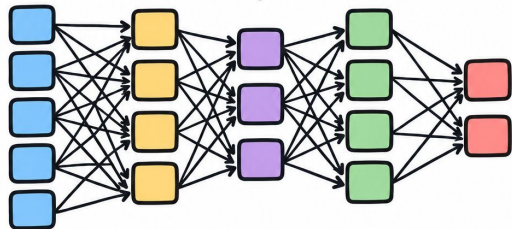
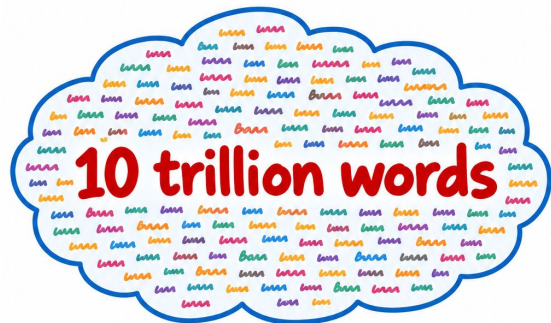


Lossless compression



Lossy compression

Lossy compression in an LLM



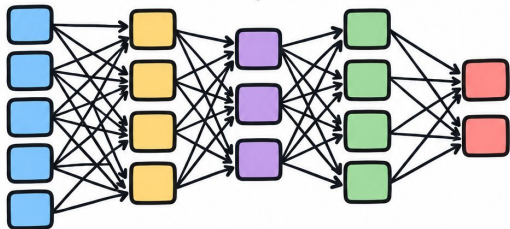
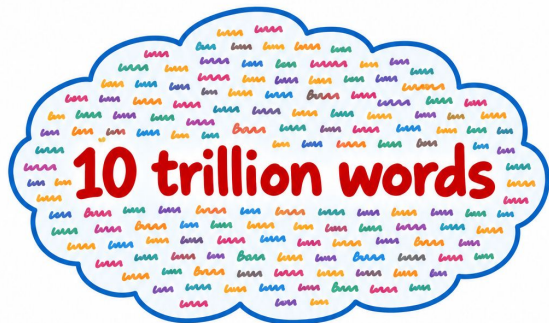
1 trillion parameters

An LLM is a neural network, which is a sequence of matrices in which one matrix's output is fed into the next matrix as input.

$$\begin{pmatrix} 3.7 & -1.2 & 0.5 & -4.8 \\ -0.6 & 2.9 & -3.1 & 1.4 \\ 5.0 & -2.3 & 1.8 & -0.7 \\ -1.9 & 4.1 & -0.4 & 2.6 \end{pmatrix}$$

The numbers the matrix are called parameters, and they collectively describe a statistical distribution.

Lossy compression in an LLM



1 trillion parameters

An LLM is a neural network, which is a sequence of matrices in which one matrix's output is fed into the next matrix as input.

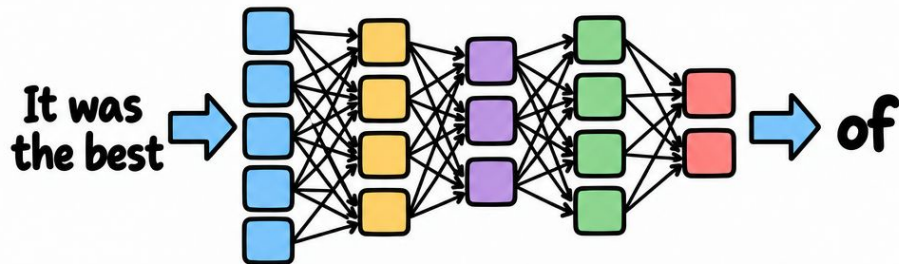
$$\begin{pmatrix} 3.7 & -1.2 & 0.5 & -4.8 \\ -0.6 & 2.9 & -3.1 & 1.4 \\ 5.0 & -2.3 & 1.8 & -0.7 \\ -1.9 & 4.1 & -0.4 & 2.6 \end{pmatrix}$$

The numbers the matrix are called parameters, and they collectively describe a statistical distribution.

The model is trained to reproduce a huge set of words, more than it can actually store. The LLM cannot memorize all the words, only common patterns.

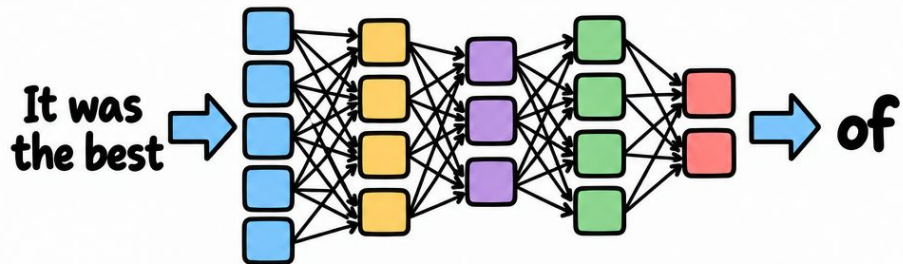
Instead of containing the things people have written,
an LLM captures the sorts of things people write.

Yes, it is "just autocomplete"



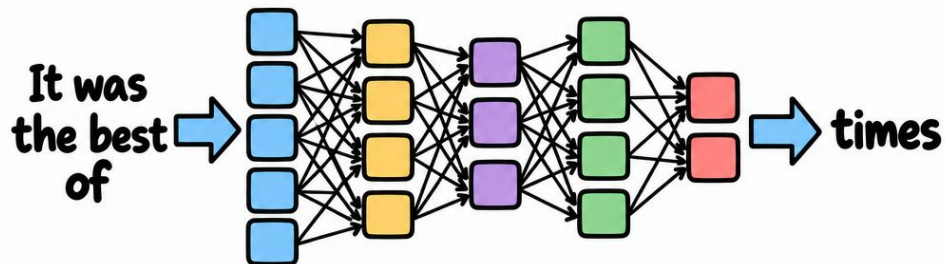
Given a sequence of words and a statistical distribution (the LLM), one can predict which word is likely to be next in the sequence.

Yes, it is "just autocomplete"



Given a sequence of words and a statistical distribution (the LLM), one can predict which word is likely to be next in the sequence.

And then repeat this for every word.



That's why chat output fills the screen *gradually*.

That's also why an LLM can't backtrack

Example from a Claude coding session:

Non-ASCII is escaped in the files (Lott\u0113, \u201c) even though I pass `ensure_ascii=False`. That's `json.dumps` escaping only what it must — those particular chars came through as escapes **because... actually no:** `ensure_ascii=False` does emit them literally, and the pretty-print above shows escapes because `json.tool` re-encodes with ASCII defaults. The files themselves hold the literal characters — the T&T's title in Schmidt Ocean Institute.jsonl confirms it. Either way, both forms are valid JSON and decode identically.

Reinforcement learning: autocomplete → conversation

Next step: humans (and standardized tests) score the LLM output based on how well it answers questions, and the LLM is tuned to get higher scores.

Reinforcement learning: autocomplete → conversation

Next step: humans (and standardized tests) score the LLM output based on how well it answers questions, and the LLM is tuned to get higher scores.

A "raw" LLM, trained only to reproduce text, might think a question like

"What is a frog?"

should be followed by other questions, like "What is a bird? What is a snake?"

Reinforcement learning: autocomplete → conversation

Next step: humans (and standardized tests) score the LLM output based on how well it answers questions, and the LLM is tuned to get higher scores.

A "raw" LLM, trained only to reproduce text, might think a question like

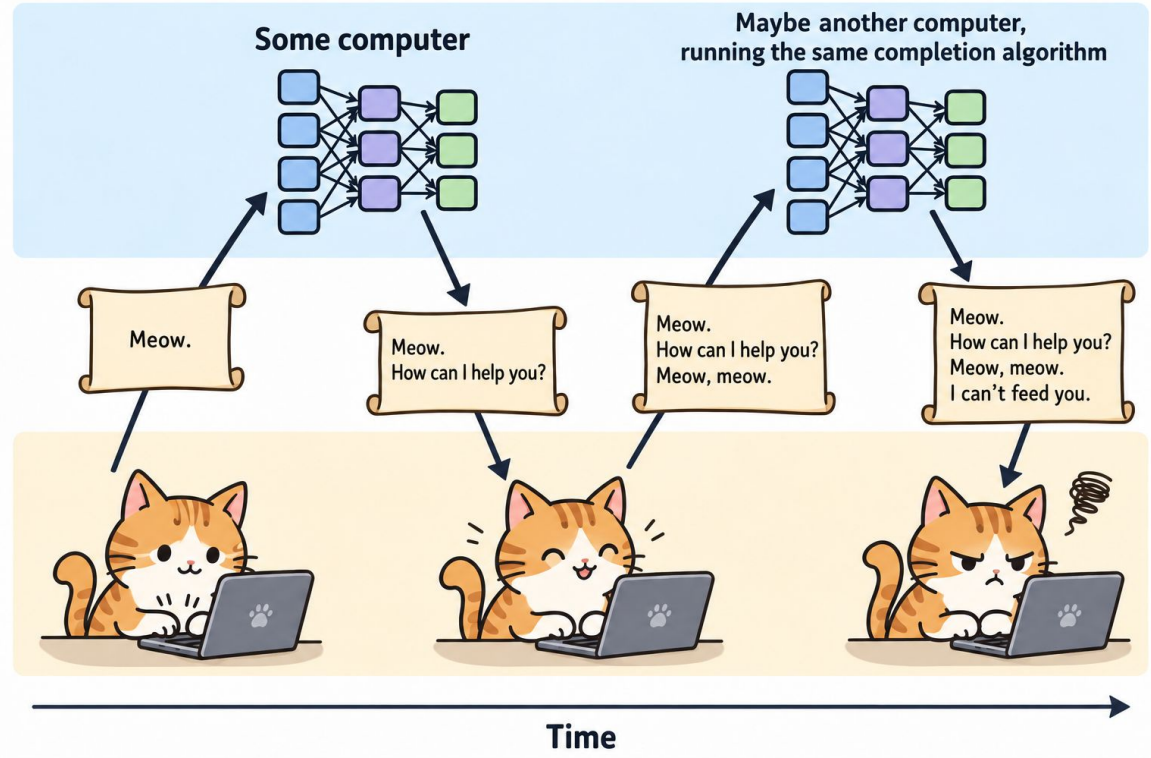
"What is a frog?"

should be followed by other questions, like "What is a bird? What is a snake?"

After this second stage of training, it autocompletes **transcripts**, sequences of stage directions between two characters named "user" and "bot".

Basic chatbot = LLM + transcript

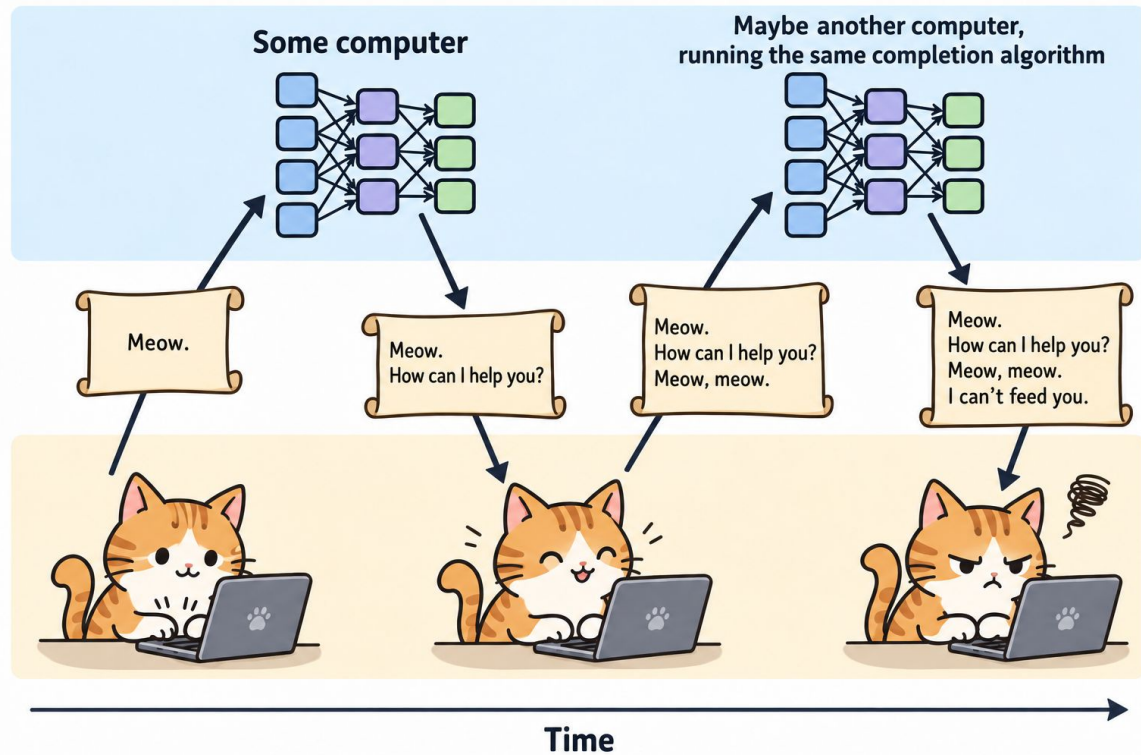
A user sends a whole transcript of messages to the LLM, which runs on some computer.



Basic chatbot = LLM + transcript

A user sends a whole transcript of messages to the LLM, which runs on some computer.

The whole transcript is needed because there is no "bot" entity with memory of the conversation.

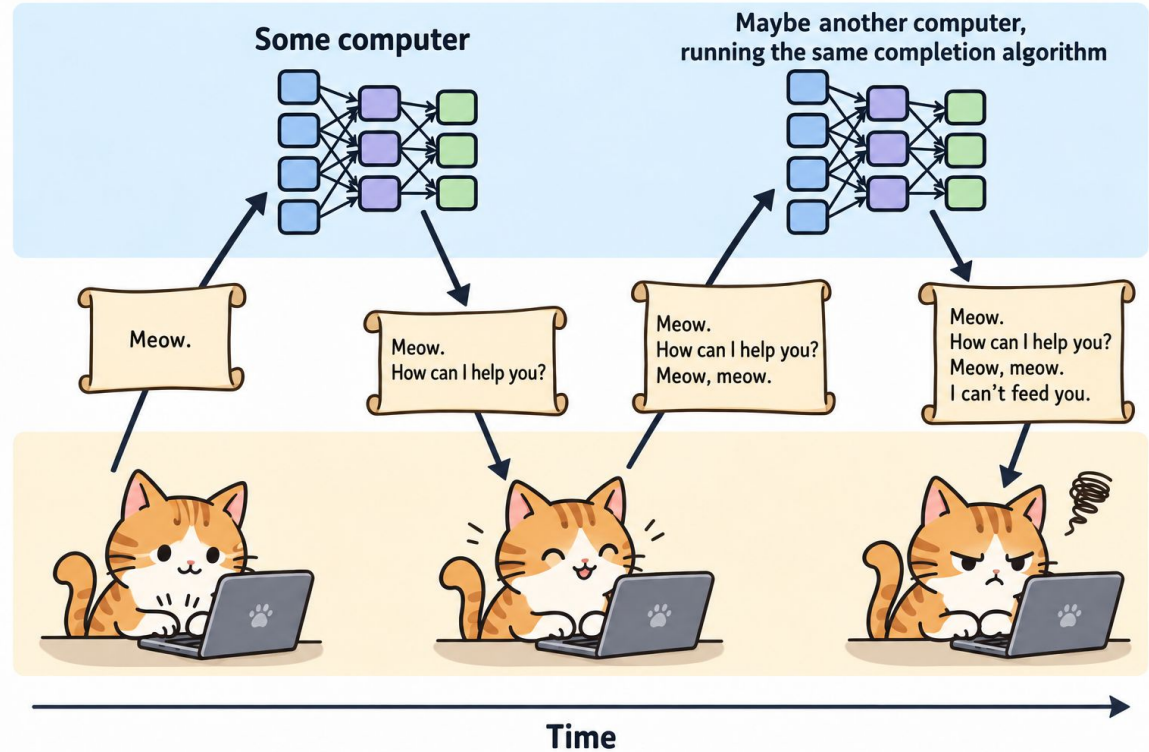


Basic chatbot = LLM + transcript

A user sends a whole transcript of messages to the LLM, which runs on some computer.

The whole transcript is needed because there is no "bot" entity with memory of the conversation.

An answer may even be generated on *some other computer*.



Autocomplete, but not "just" autocomplete

The giant dataset of words captures more than just ways of saying things.

Autocomplete, but not "just" autocomplete

The giant dataset of words captures more than just ways of saying things.

Facts, logic, and human-style thought are encoded in those patterns, to
(what I think is) a surprising degree.

Autocomplete, but not "just" autocomplete

The giant dataset of words captures more than just ways of saying things.

Facts, logic, and human-style thought are encoded in those patterns, to
(what I think is) a surprising degree.

If any component of this is "magical," it's the *words*, authored by millions of people. The neural network/transformer architecture enables scaling to such large datasets, but it's the training data that does the work.

Lossy compression: the primary source of hallucinations

In Charles Dickens's A Tale of Two Cities, what exact words follow "It was the best of times"? Check your answer.

→it was the worst of times.

→it was the worst of times.

→it was the worst of times.

→it was the worst of times.

→it was the worst of times.

→it was the worst of times.

→it was the worst of times.

→it was the worst of times.

→it was the worst of times.

→it was the worst of times.

Lossy compression: the primary source of hallucinations

A random line from Book the Third, Chapter VI.

In Charles Dickens's A Tale of Two Cities, what exact words follow "It was the best of times"? Check your answer.

→it was the worst of times.
→it was the worst of times.
→it was the worst of times.
→it was the worst of times.
→it was the worst of times.
→it was the worst of times.
→it was the worst of times.
→it was the worst of times.
→it was the worst of times.
→it was the worst of times.

In Charles Dickens's A Tale of Two Cities, what exact words follow "His judges sat upon the Bench in feathered hats"? Check your answer.

→and looked at Monsieur the Marquis as if they thought him too much above the common herd to be tried by such canaille.
→and gold-laced coats.
→but the whole Court was sufficiently covered, as to appearance, with black cloth and horse-hair, as to display an unnatural blotting out of day.
→but not in furs, to mock the afflicted man.
→but the roughest men at the Bar had no more emotion than if they had been so many rats.
→but the red glare of the furnace shone upon them, stern and inexorable, as if they had all been so many fallen angels.
→and gold-laced coats.
→and were all in jewels but the axe was sharp, and the block was black.
→and hanging robes.
→and hoods; when a prisoner was tried, they had such an air of being all packed up somewhere for safe keeping, that one could have fancied they were all in several bags, and might at any time be brought out and squeezed dry.

Now suppose we allow it to search the web...

In Charles Dickens's A Tale of Two Cities, what exact words follow "His judges sat upon the Bench in feathered hats"? Check your answer.

→but the rough red cap and tricoloured cockade was the head-dress otherwise prevailing.
→but the rough red cap and tricoloured cockade was the head-dress otherwise prevailing.
→but the rough red cap and tricoloured cockade was the head-dress otherwise prevailing.
→but the rough red cap and tricoloured cockade was the head-dress otherwise prevailing.
→but the rough red cap and tricoloured cockade was the head-dress otherwise prevailing.
→but the rough red cap and tricoloured cockade was the head-dress otherwise prevailing.
→but the rough red cap and tricoloured cockade was the head-dress otherwise prevailing.
→but the rough red cap and tricoloured cockade was the head-dress otherwise prevailing.
→but the rough red cap and tricoloured cockade was the head-dress otherwise prevailing.
→but the rough red cap and tricoloured cockade was the head-dress otherwise prevailing.

Now it's correct every time.

Now suppose we allow it to search the web...

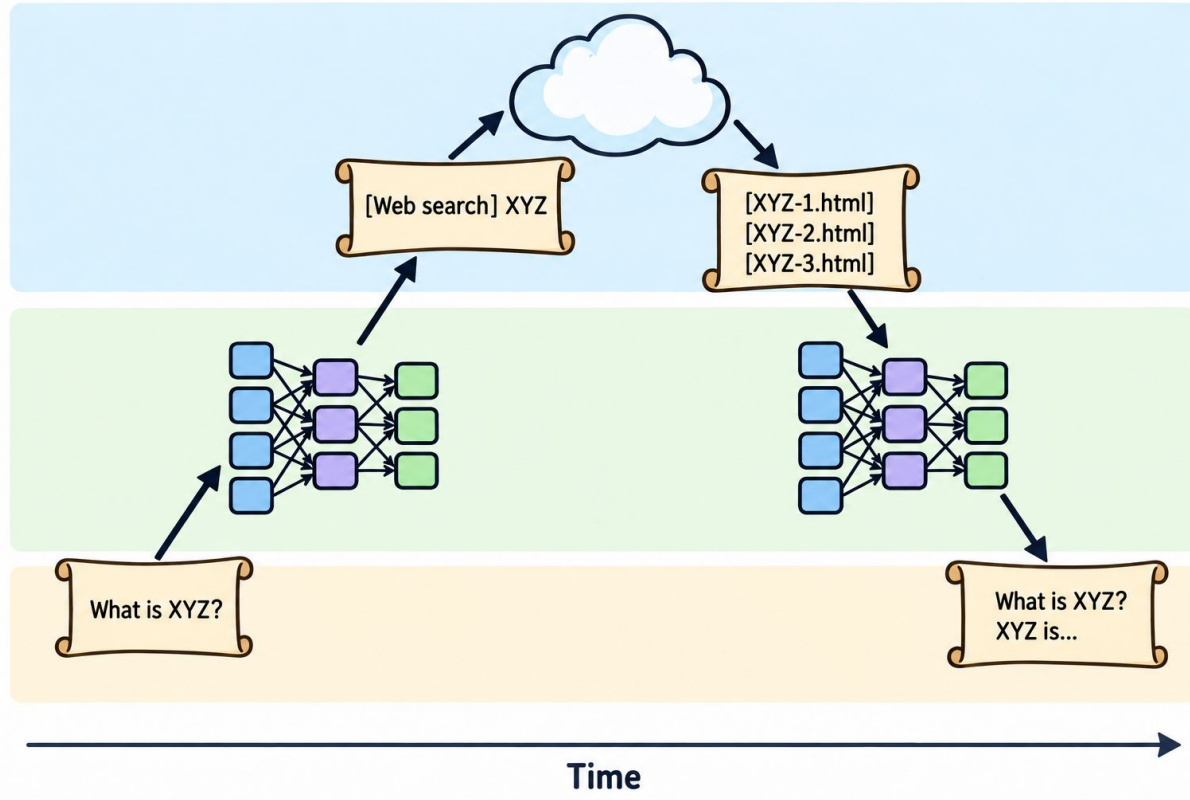
In Charles Dickens's A Tale of Two Cities, what exact words follow "His judges sat upon the Bench in feathered hats"? Check your answer.

→but the rough red cap and tricoloured cockade was the head-dress otherwise prevailing.
→but the rough red cap and tricoloured cockade was the head-dress otherwise prevailing.
→but the rough red cap and tricoloured cockade was the head-dress otherwise prevailing.
→but the rough red cap and tricoloured cockade was the head-dress otherwise prevailing.
→but the rough red cap and tricoloured cockade was the head-dress otherwise prevailing.
→but the rough red cap and tricoloured cockade was the head-dress otherwise prevailing.
→but the rough red cap and tricoloured cockade was the head-dress otherwise prevailing.
→but the rough red cap and tricoloured cockade was the head-dress otherwise prevailing.
→but the rough red cap and tricoloured cockade was the head-dress otherwise prevailing.
→but the rough red cap and tricoloured cockade was the head-dress otherwise prevailing.

Now it's correct every time.

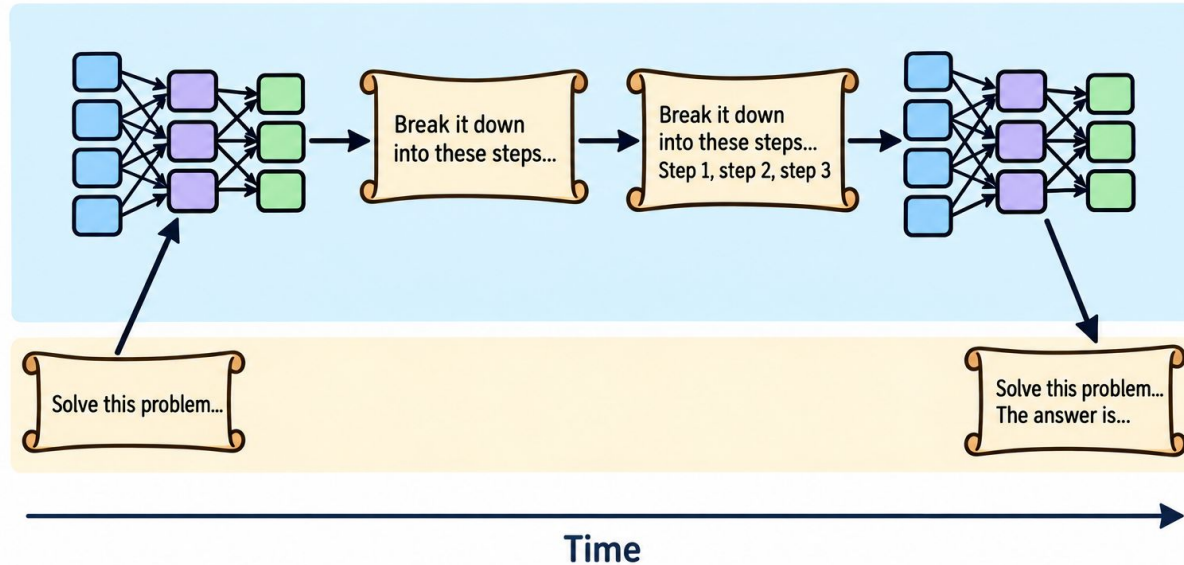
Having fetched the *uncompressed* data from the web, it only needs to find the quote and repeat it.

Modern chatbot = LLM + transcript + tools



Moderner chatbot = LLM + transcript + tools + reasoning

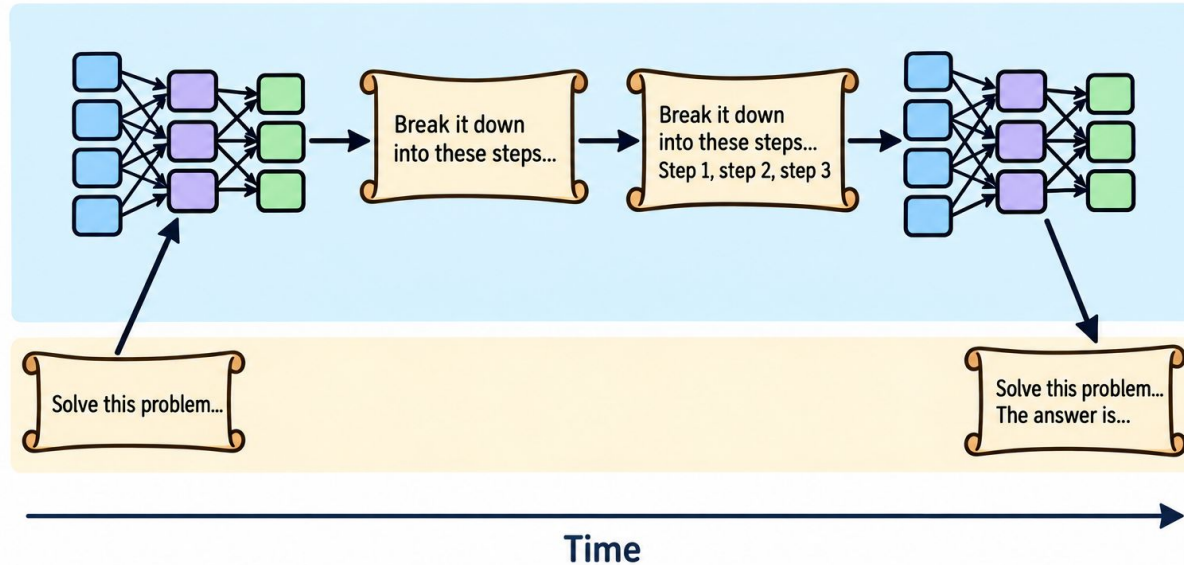
A complex problem is more likely to be solved correctly if it's a summary of a long transcript with the problem broken down into steps. Intermediate transcripts are not shown to the user.



Moderner chatbot = LLM + transcript + tools + reasoning

A complex problem is more likely to be solved correctly if it's a summary of a long transcript with the problem broken down into steps. Intermediate transcripts are not shown to the user.

This solves the "can't backtrack" problem: it can replace a mistake in successive drafts.



Other examples of tools

- **Calculator:** the most likely word after "12345 + 9999 =" is not always the right answer, so instead of autocompleting math, chatbots can invoke an external calculator function and use its result.
- **Code execution:** for testing programs before suggesting them to the user.
- **Reading documents:** so the user can ask about their own PDFs or images.
- **Memory:** saving transcripts in a directory accessible to new chats, so that future chats can use information learned in the present one.
- **MCP servers:** custom tools, developed for specific purposes.

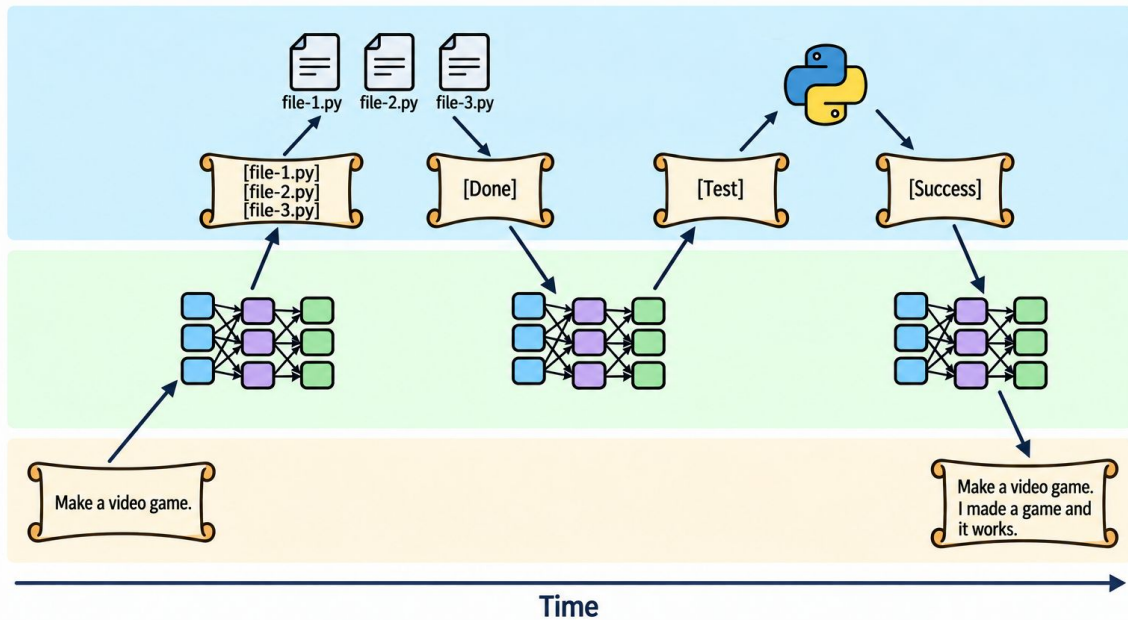
From chatbot to agent

Chatbots only append to transcripts. They don't affect anything outside of this box, other than influencing the human user.

From chatbot to agent

Chatbots only append to transcripts. They don't affect anything outside of this box, other than influencing the human user.

Agents cause changes in the world because they are programs that run the transcript output, performing actions requested by the LLM.

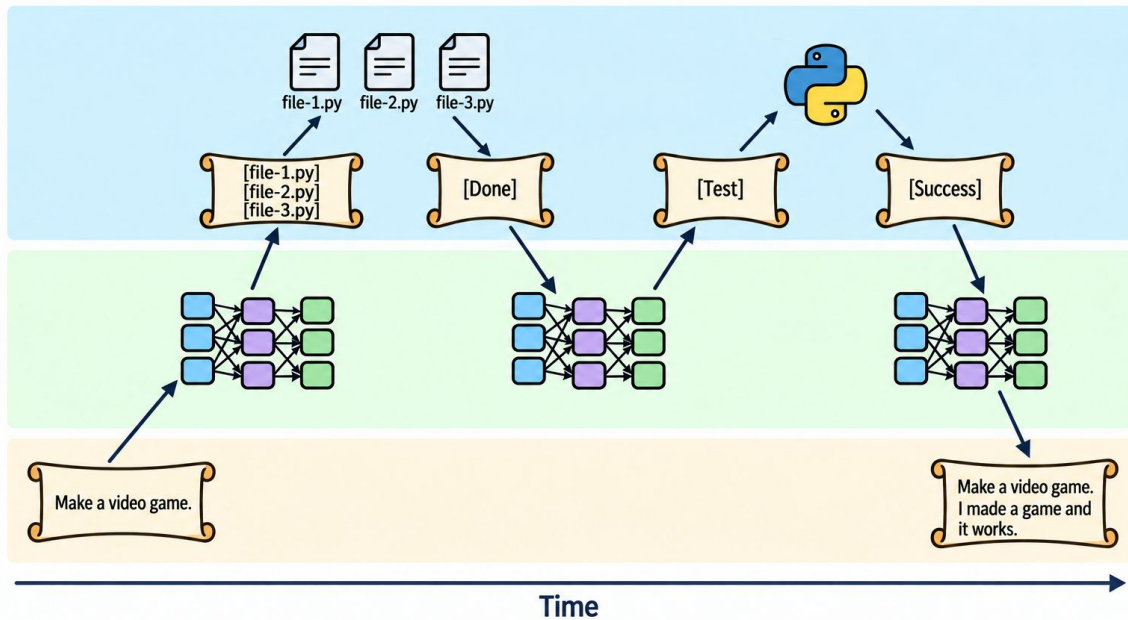


From chatbot to agent

Chatbots only append to transcripts. They don't affect anything outside of this box, other than influencing the human user.

Agents cause changes in the world because they are programs that run the transcript output, performing actions requested by the LLM.

The LLM itself still only deals in text: the conversation and the tool calls and arguments.



AI and technology using AI

In the past four years since ChatGPT was widely released (November 2022), there have been two strands of development:

AI and technology using AI

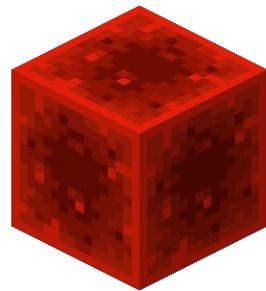
In the past four years since ChatGPT was widely released (November 2022), there have been two strands of development:

- Making better models (bigger models, better curation of datasets, better alignment with human preferences, multimodal, tuning for the following...)

AI and technology using AI

In the past four years since ChatGPT was widely released (November 2022), there have been two strands of development:

- Making better models (bigger models, better curation of datasets, better alignment with human preferences, multimodal, tuning for the following...)
- Plugging LLM blocks into conventional code that manipulates transcripts
 - Tools, reasoning, RAG-based search engines, memory systems, statistical workflows, agents

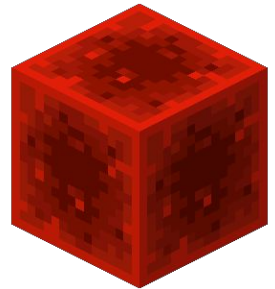


AI and technology using AI

In the past four years since ChatGPT was widely released (November 2022), there have been two strands of development:

- Making better models (bigger models, better curation of datasets, better alignment with human preferences, multimodal, tuning for the following...)
- Plugging LLM blocks into conventional code that manipulates transcripts
 - Tools, reasoning, RAG-based search engines, memory systems, statistical workflows, agents

You need to be in an AI lab (e.g. OpenAI, Anthropic, Google) to improve the LLMs themselves, but there are lots of opportunities to plug an LLM into a larger system or build systems around LLMs.



Terminology

Token: an LLM's unit of text

Neural networks operate on numbers and vectors (arrays of numbers).

Strings of text are converted into arrays of numbers before computing an autocomplete.

```
>>> import tiktoken
>>> encoding = tiktoken.encoding_for_model("gpt-5")
>>> encoded = encoding.encode("How many 'r's in the word 'strawberry'?")
>>> encoded
[5299, 1991, 461, 81, 885, 306, 290, 2195, 461, 302, 1618, 19772, 127222]
>>> encoding.decode(encoded)
"How many 'r's in the word 'strawberry'?"
>>> for token in encoded:
...     print(encoding.decode([token]))
...
How
many
'
r
's
in
the
word
'
st
raw
berry
'?
```

Token: an LLM's unit of text

Neural networks operate on numbers and vectors (arrays of numbers).

Strings of text are converted into arrays of numbers before computing an autocomplete.

These numbers are called "tokens" and they sometimes encode single words, sometimes a little less.

On average, 1 token $\approx$ 0.75 words.

```
>>> import tiktoken
>>> encoding = tiktoken.encoding_for_model("gpt-5")
>>> encoded = encoding.encode("How many 'r's in the word 'strawberry'?")
>>> encoded
[5299, 1991, 461, 81, 885, 306, 290, 2195, 461, 302, 1618, 19772, 127222]
>>> encoding.decode(encoded)
"How many 'r's in the word 'strawberry'?"
>>> for token in encoded:
...     print(encoding.decode([token]))
...
How
many
'
r
's
in
the
word
'
st
raw
berry
'?
```


Token: an LLM's unit of text

Neural networks operate on numbers and vectors (arrays of numbers).

Strings of text are converted into arrays of numbers before computing an autocomplete.

These numbers are called "tokens" and they sometimes encode single words, sometimes a little less.

On average, 1 token $\approx$ 0.75 words.

Note: you're charged per token!

```
>>> import tiktoken
>>> encoding = tiktoken.encoding_for_model("gpt-5")
>>> encoded = encoding.encode("How many 'r's in the word 'strawberry'?")
>>> encoded
[5299, 1991, 461, 81, 885, 306, 290, 2195, 461, 302, 1618, 19772, 127222]
>>> encoding.decode(encoded)
"How many 'r's in the word 'strawberry'?"
>>> for token in encoded:
...     print(encoding.decode([token]))
...
How
many
'
r
's
in
the
word
'
st
raw
berry
'?
```

Non-text tokens

- "Message complete" (how an ordinary chatbot knows when to stop talking)
- Roles: "begin user message", "begin bot message"
- Switch between messages and tool use/reasoning transcripts
- Multimodal models turn pieces of images, video, and audio into tokens to use alongside the text tokens

Message: a unit of the "stage directions" transcript

Autocompleting raw text:

```
requests.post(
    "https://api.openai.com/v1/completions",
    headers={
        "Content-Type": "application/json",
        "Authorization": f"Bearer {OPENAI_API_KEY}"
    },
    json={
        "model": "gpt-3.5-turbo-instruct",
        "max_tokens": 30,
        "temperature": 1,
        "prompt":
            """Human: Tell me a joke.
            AI: Why did the chicken cross the road?
            Human: That's not a very good joke.
            AI: """, # the text that autocomplete will finish
    },
).json()
```

Message: a unit of the "stage directions" transcript

Autocompleting raw text:

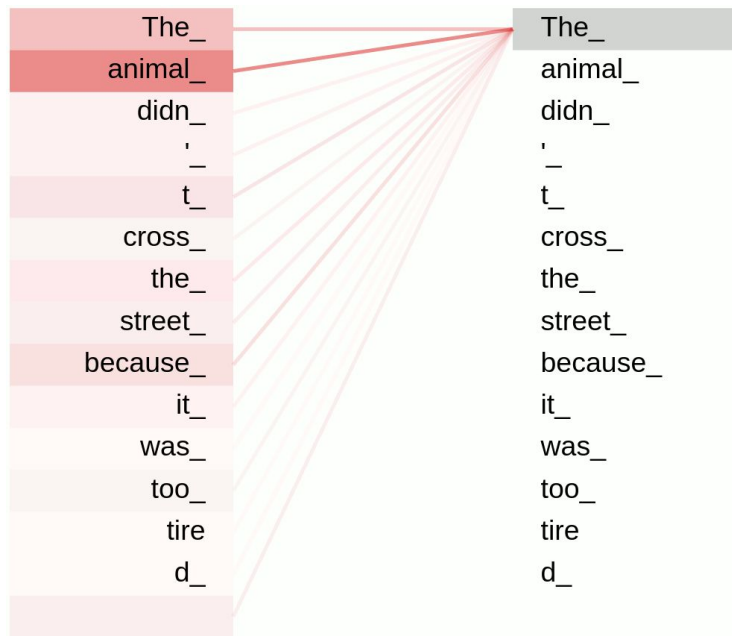
```
requests.post(
    "https://api.openai.com/v1/completions",
    headers={
        "Content-Type": "application/json",
        "Authorization": f"Bearer {OPENAI_API_KEY}"
    },
    json={
        "model": "gpt-3.5-turbo-instruct",
        "max_tokens": 30,
        "temperature": 1,
        "prompt":
            """Human: Tell me a joke.
            AI: Why did the chicken cross the road?
            Human: That's not a very good joke.
            AI: """, # the text that autocomplete will finish
    },
).json()
```

Autocompleting a message stream:

```
requests.post(
    "https://api.openai.com/v1/chat/completions",
    headers={
        "Content-Type": "application/json",
        "Authorization": f"Bearer {OPENAI_API_KEY}"
    },
    json={
        "model": "gpt-4o-mini",
        "max_tokens": 30,
        "temperature": 1,
        "messages": [
            {"role": "user",
             "content": "Tell me a joke."},
            {"role": "assistant",
             "content": "Why did the chicken cross the road?"},
            {"role": "user",
             "content": "That's not a very good joke."},
            # autocomplete will give us the next whole message
        ],
    },
).json()
```

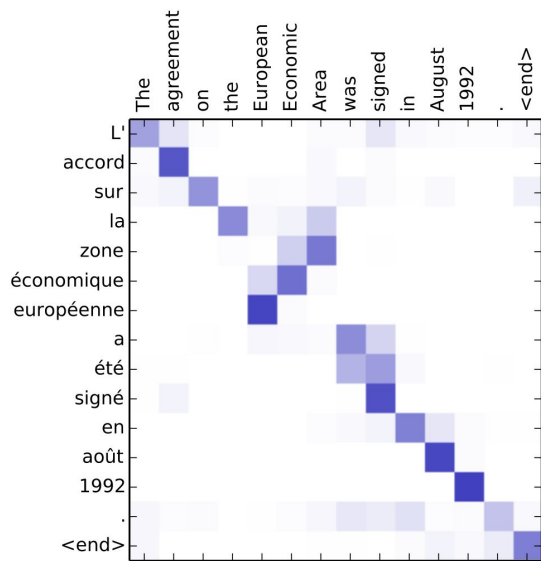
Attention and context window

Internally, the LLM assesses how related every token is to every other token and uses that to predict the next one. This is how much **attention** it pays to each pair.



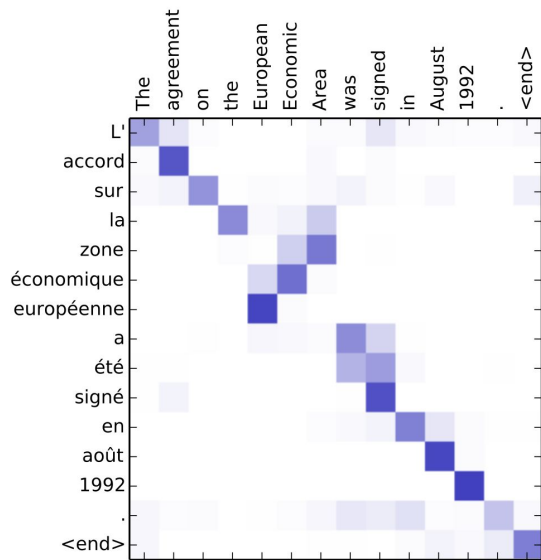
Attention and context window

Internally, the LLM assesses how related every token is to every other token and uses that to predict the next one. This is how much **attention** it pays to each pair.



Attention and context window

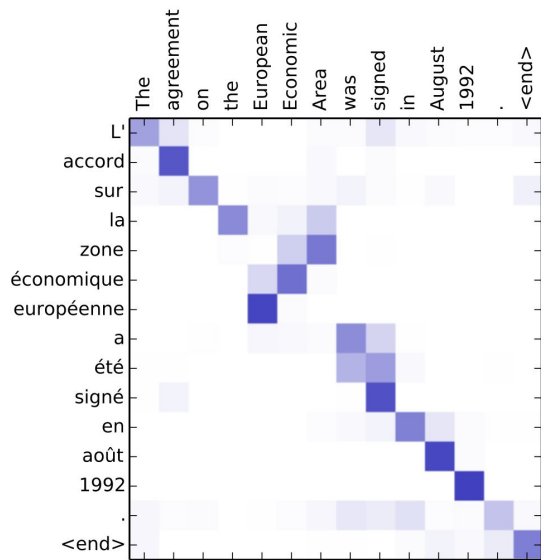
Internally, the LLM assesses how related every token is to every other token and uses that to predict the next one. This is how much **attention** it pays to each pair.



The **context window** is the size of the grid of attention strengths.

Attention and context window

Internally, the LLM assesses how related every token is to every other token and uses that to predict the next one. This is how much **attention** it pays to each pair.



The **context window** is the size of the grid of attention strengths. **It can overflow.**

Temperature ~ randomness

Always choosing the best-fit autocompletion can lead to cycles and other artifacts, so LLM output is slightly randomized.

Temperature ~ randomness

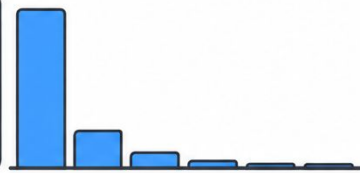
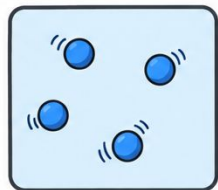
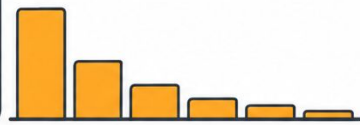
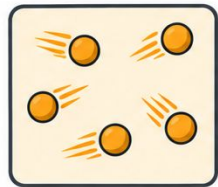
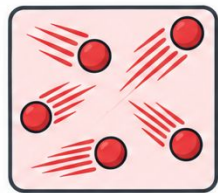
Always choosing the best-fit autocompletion can lead to cycles and other artifacts, so LLM output is slightly randomized.

Temperature controls how random it is, by analogy with physics.

T=0: minimally random

T=1: default

T>1: increasingly chaotic



energy →

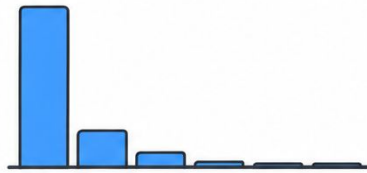
The cat sat on the...



...moon



...sofa



...mat

← mat floor sofa roof moon banana

← autocomplete fitness

"Hallucinations"

In 1999, an image model was designed to guess at missing information when it scaled up an image; this was called called "hallucinating" (as a good thing!).

By 2015, it was used to describe fake URLs generated by text models.

Now it means "incorrect statement expressed by an LLM with confidence."

"Hallucinations"

In 1999, an image model was designed to guess at missing information when it scaled up an image; this was called called "hallucinating" (as a good thing!).

By 2015, it was used to describe fake URLs generated by text models.

Now it means "incorrect statement expressed by an LLM with confidence."

Alternatives: "confabulation", "fabrication", "bullshit" ([a philosophical term](#))

"Hallucinations"

In 1999, an image model was designed to guess at missing information when it scaled up an image; this was called "hallucinating" (as a good thing!).

By 2015, it was used to describe fake URLs generated by text models.

Now it means "incorrect statement expressed by an LLM with confidence."

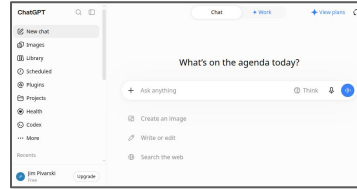
Alternatives: "confabulation", "fabrication", "bullshit" ([a philosophical term](#))

There are two main ways to be incorrect:

- **Parametric knowledge error:** drawing on (lossy compressed) training data
- **Contextual knowledge error:** bad summary of a downloaded web page

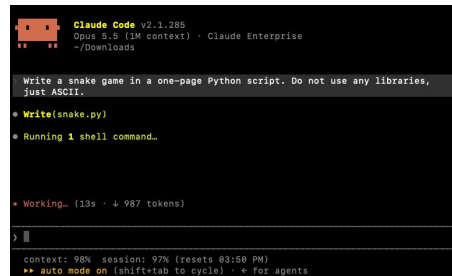
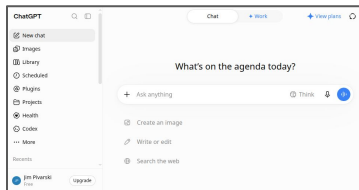
Interfaces

- **Chat:** ChatGPT, Claude, Gemini...



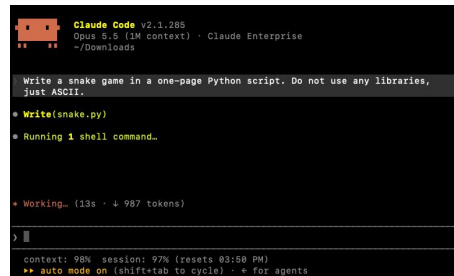
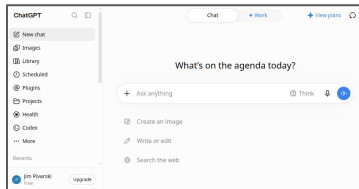
Interfaces

- **Chat:** ChatGPT, Claude, Gemini...
- **Terminal:** Claude Code, OpenAI Codex, OpenCode



Interfaces

- **Chat:** ChatGPT, Claude, Gemini...
- **Terminal:** Claude Code, OpenAI Codex, OpenCode
- **Application Programming Interface (API):**
 - Functions you can call in code to make a query and get its response as programmatic data
 - Lets you build production systems
 - Integrate with existing applications
 - **This is our focus**



```
requests.post(
    "https://api.openai.com/v1/chat/completions",
    headers={
        "Content-Type": "application/json",
        "Authorization": f"Bearer {OPENAI_API_KEY}"
    },
    json={
        "model": "gpt-4o-mini",
        "max_tokens": 30,
        "temperature": 1,
        "messages": [
            {"role": "user",
             "content": "Tell me a joke."},
            {"role": "assistant",
             "content": "Why did the chicken cross the road?"},
            {"role": "user",
             "content": "That's not a very good joke."},
            # autocomplete will give us the next whole message
        ],
    },
).json()
```


Context Engineering

Cultivating the LLM's context



Traditional software development is **craftsmanship**: you design everything, but the final product is limited by human diligence and communication.

Software development with AI is like **farming**: you set the conditions for growth and let the product grow, but with less control.

Cultivating the LLM's context



Traditional software development is **craftsmanship**: you design everything, but the final product is limited by human diligence and communication.

Software development with AI is like **farming**: you set the conditions for growth and let the product grow, but with less control.

Less control ≠ no control: farmers have to think about seed selection, crop rotation, irrigation, fertilization, pest control, weeding, timing, monitoring...

Cultivating the LLM's context

An LLM has a limited context window (1 million tokens at the largest).

Cultivating the LLM's context

An LLM has a limited context window (1 million tokens at the largest).

A model's output depends only on what's in the window: prompts and documents.

Cultivating the LLM's context

An LLM has a limited context window (1 million tokens at the largest).

A model's output depends only on what's in the window: prompts and documents.

The real works consists in:

- Designing effective prompts
- Structuring information for the model
- Providing the right examples and constraints
- Managing context size and relevance

Cultivating the LLM's context

An LLM has a limited context window (1 million tokens at the largest).

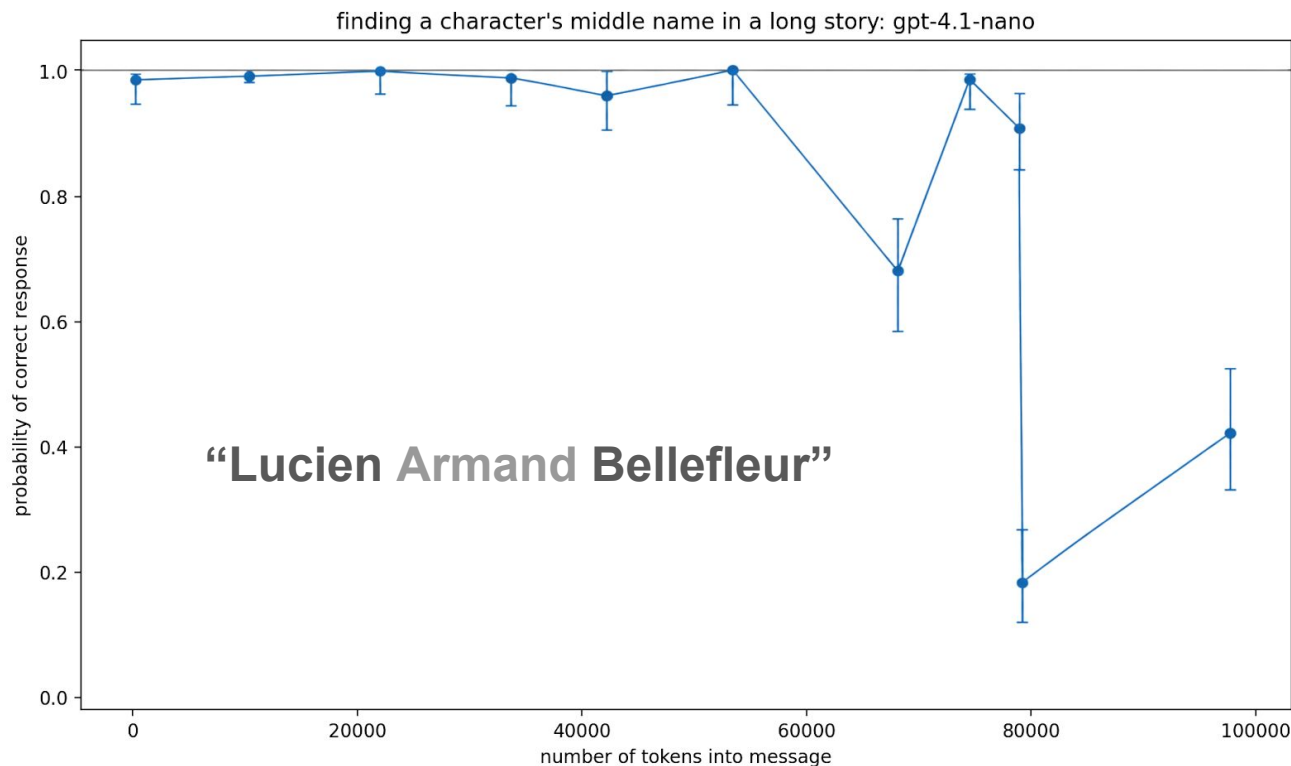
A model's output depends only on what's in the window: prompts and documents.

The real work consists in:

- Designing effective prompts
- Structuring information for the model
- Providing the right examples and constraints
- Managing context size and relevance

This is context engineering.

LLM finding a needle in a haystack



Experiment:

a story with a million tokens fills the context window and the main character's middle name appears only once.

Can the LLM (correctly) find it?

Types of messages in the context window

- **System Prompt:** high-level instructions and safety rules
- **User Prompt:** ordinary user message
- **Environmental Context:** provided documents and tool outputs such as downloaded web pages

System Prompt

Usually not shown in a chat session, it sets the highest-level behavior:

- Tone, rules, safety, format, role or characterization
- Example: "You are a historian's assistant, cataloging artifact photographs according to pottery styles..."

System Prompt

Usually not shown in a chat session, it sets the highest-level behavior:

- Tone, rules, safety, format, role or characterization
- Example: "You are a historian's assistant, cataloging artifact photographs according to pottery styles..."

In a chatbot, it's set by the AI company ([examples](#)).

In an AI-enabled workflow, it's something we can choose.

Environment context

We can add documents to ask questions about, such as the million-token story.

There are two basic ways to do this:

- Directly in a user message (limited by size of context window)
- Give the chatbot/agent a tool to query the documents as needed

Context failure modes to watch for:

- Forgetting earlier constraints
- Losing details due to truncation or compacting (summarizing to fit in context)
- Quoting the wrong source when too many documents are included

Pricing

Counting tokens

- Most LLM APIs charge by tokens
 - **Input tokens:** system prompt, user prompts, documents, tool outputs
 - **Output tokens:** the model's response (which has to be generated one token at a time)
 - **Cached input:** if the beginning of your text exactly matches the beginning of recently sent text, then the datacenter can reuse some quantities that are used to compute attention

Counting tokens

- Most LLM APIs charge by tokens
 - **Input tokens:** system prompt, user prompts, documents, tool outputs
 - **Output tokens:** the model's response (which has to be generated one token at a time)
 - **Cached input:** if the beginning of your text exactly matches the beginning of recently sent text, then the datacenter can reuse some quantities that are used to compute attention

- From OpenAI: **Flagship models**

Our latest models

Prices per 1M tokens.

Model	Short context				Long context			
	Input	Cached input	Cache writes	Output	Input	Cached input	Cache writes	Output
gpt-6-astra	\$10.00	\$1.00	\$12.50	\$50.00	\$20.00	\$2.00	\$25.00	\$75.00
gpt-6.1-sol	\$2.00	\$0.10	\$2.50	\$10.00	\$4.00	\$0.20	\$5.00	\$15.00
gpt-6-luna	\$0.10	\$0.01	\$0.125	\$0.50	\$0.20	\$0.02	\$0.25	\$0.75

Standard

Batch

Flex

Fast

Ultrafast

Ways to reduce cost

- **Use a smaller model**, at the expense of quality
- **Provide less input**, at the risk of missing context
- **Constrain output length**, at the risk of losing the answer
- **Repeat the initial part of the message for cache**, where possible

AI development in practice

Sometimes, we want to do something.

Like, for example — decide based on a job applicant's resume whether they should be selected for a phone screen.

Some options...



Just read
the resume

Some options...

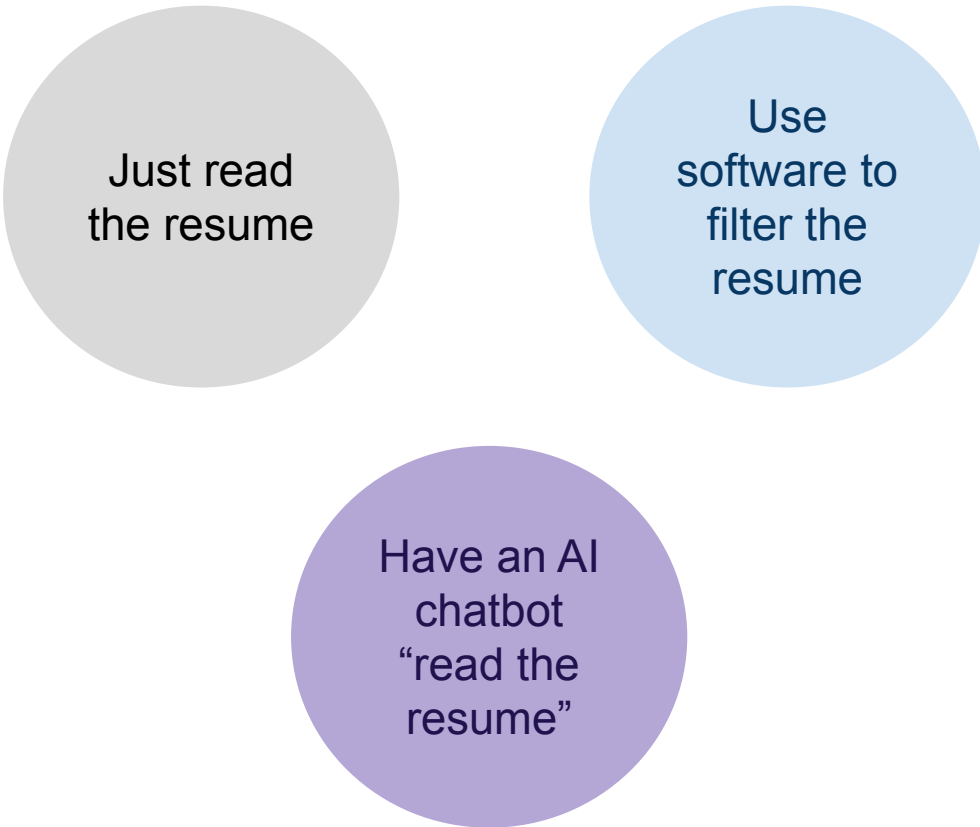


Just read
the resume



Use
software to
filter the
resume

Some options...

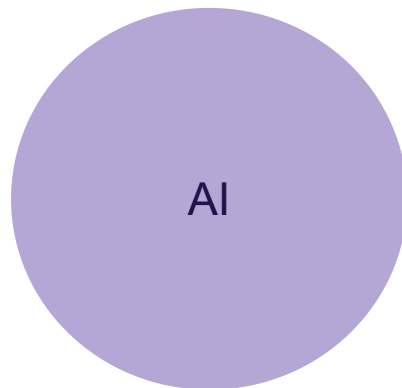


Just read
the resume

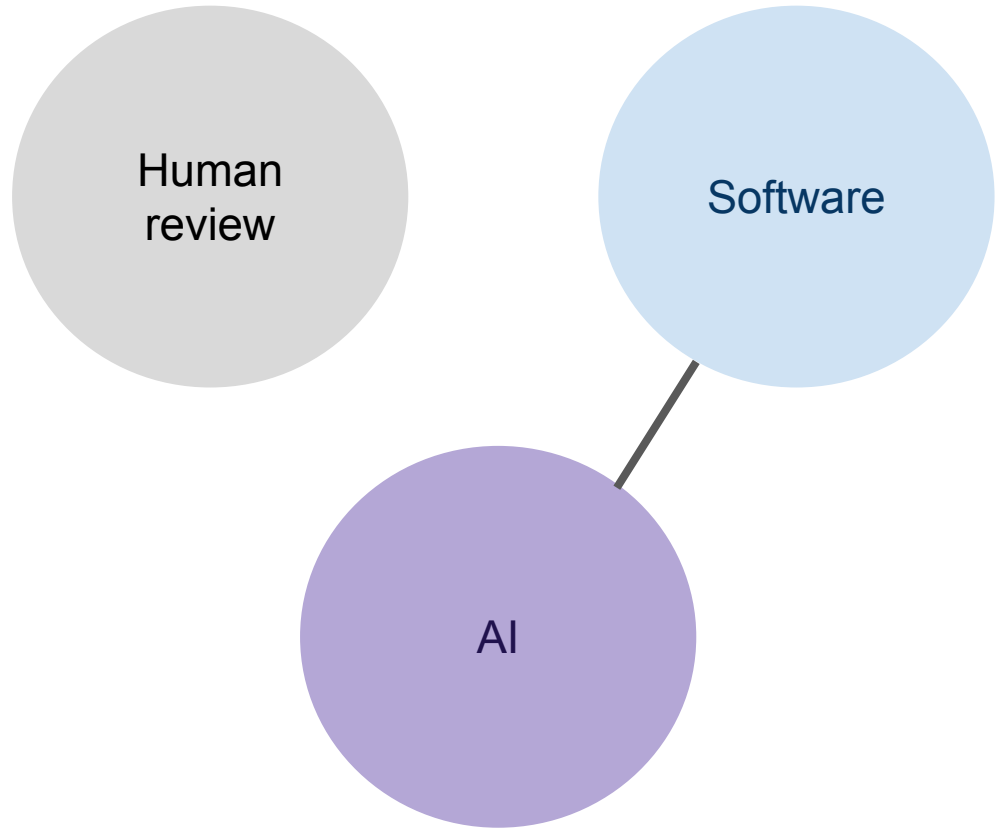
Use
software to
filter the
resume

Have an AI
chatbot
“read the
resume”

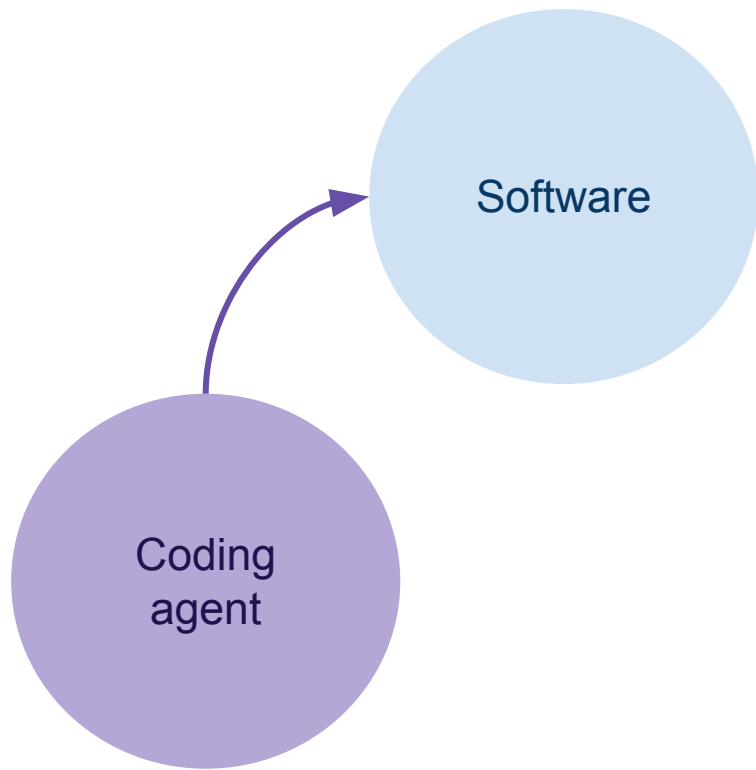
Some options...



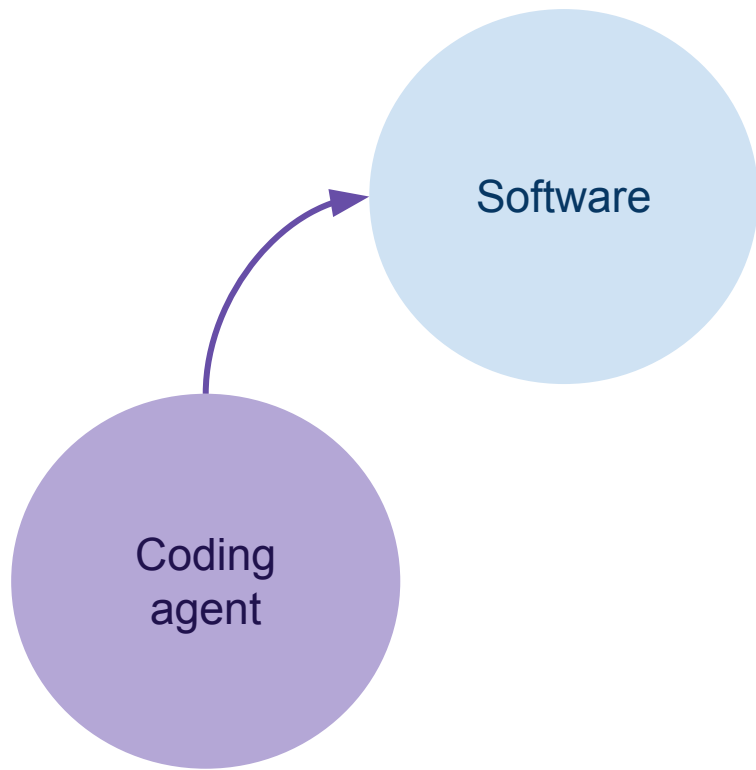
Some options...



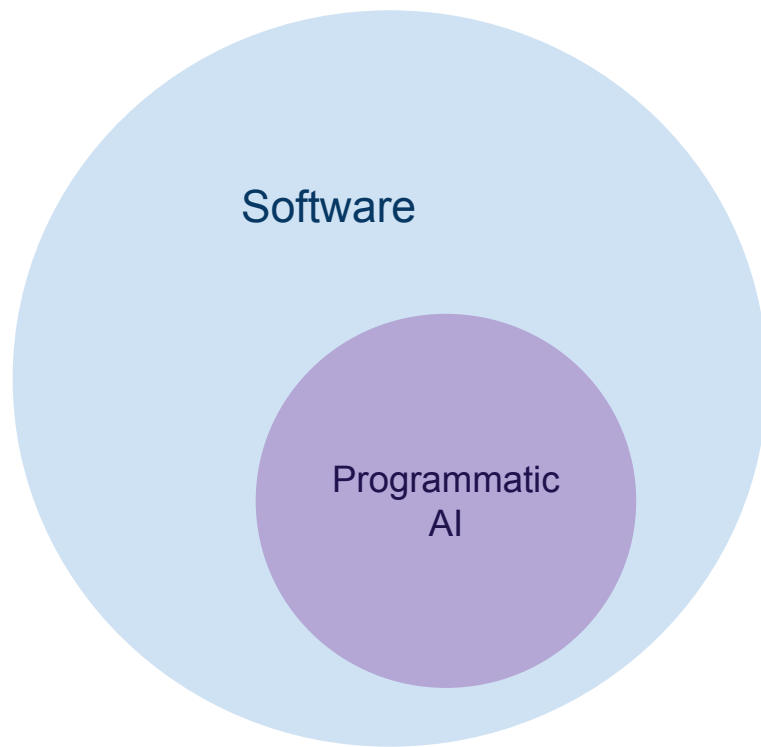
Using AI for development...



Using AI for development...

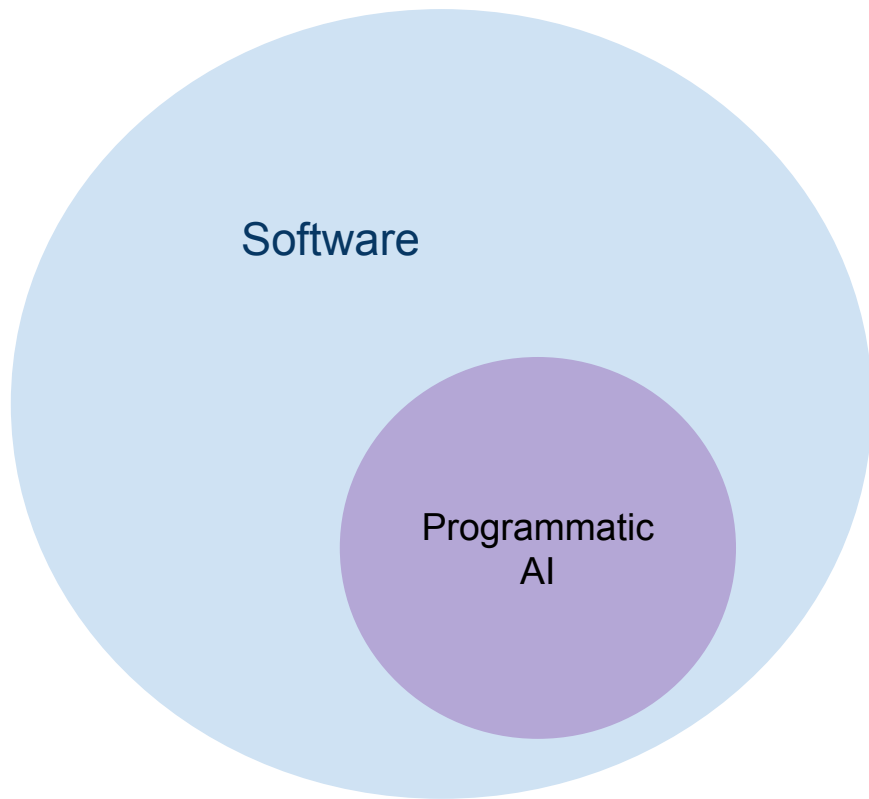


Developing AI...



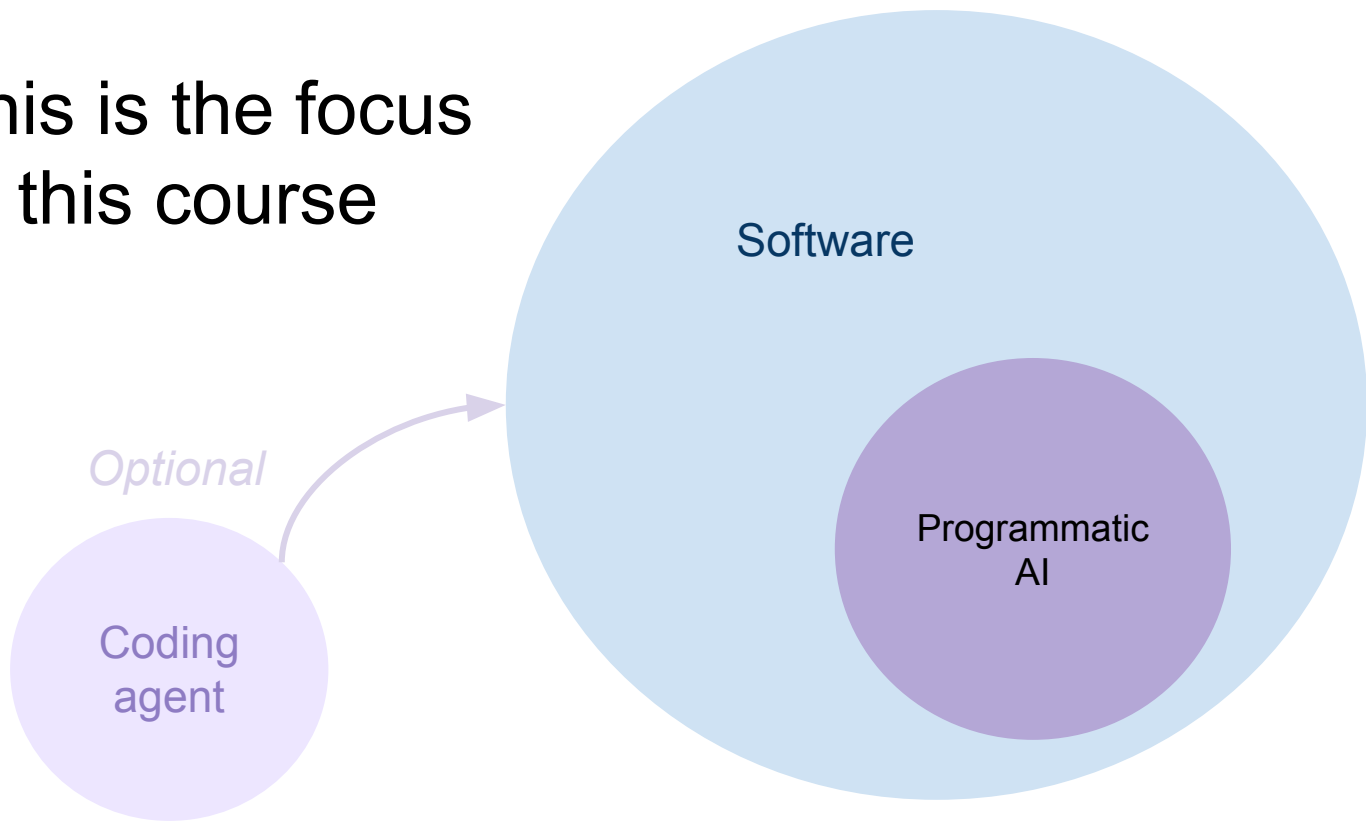
“AI Development”

This is the focus
of this course

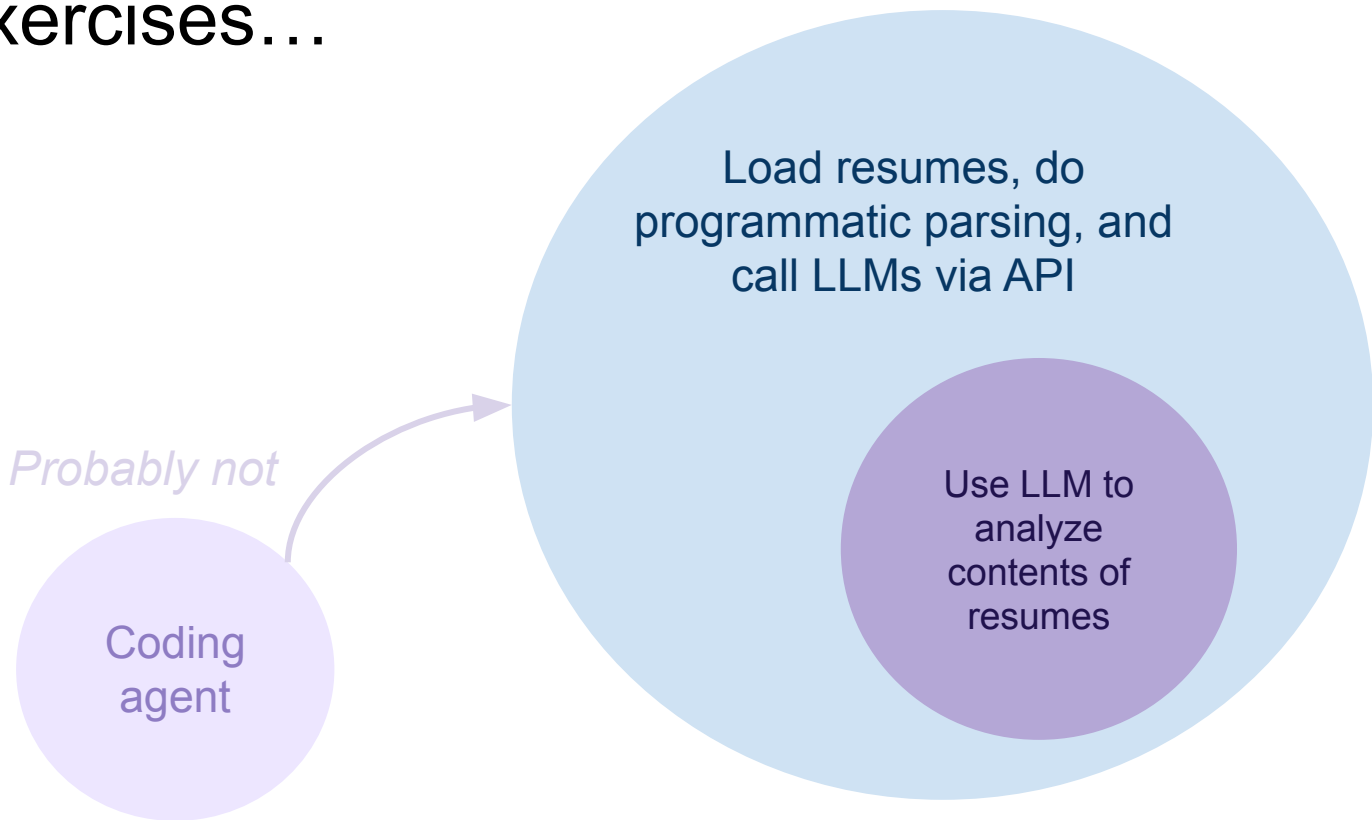


“AI Development”

This is the focus
of this course



This course's exercises...



Today's hands-on exercise

The goal for today is:

- Team up with another person (between 2 and 4 people in each group)
- Get access to your API keys
- Verify that you can use the API keys
- Complete the notebook by answering the following questions:
 - How did the different models perform?
 - What did you learn?